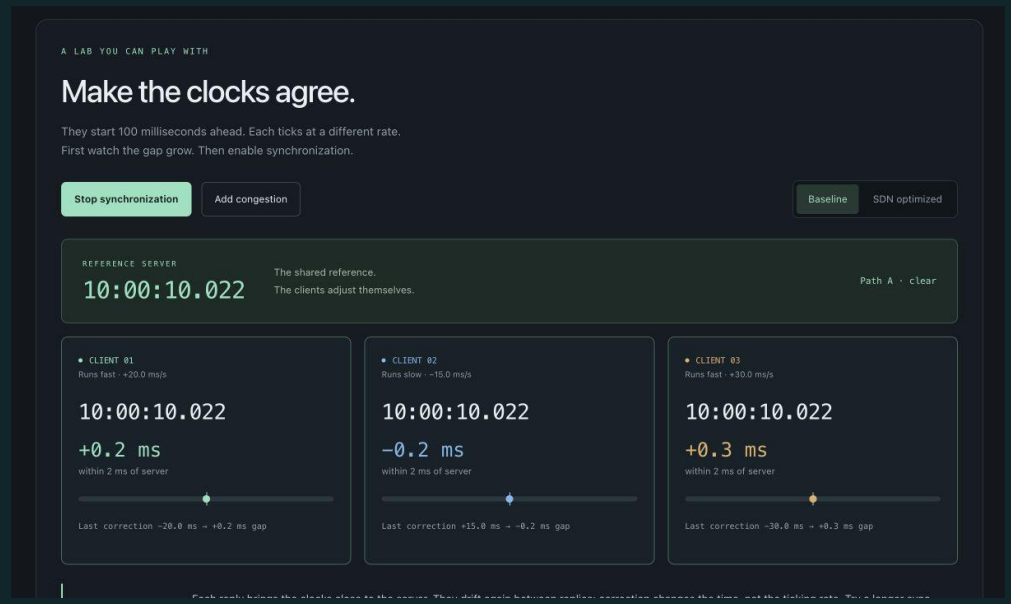
TimeSync

Clock sync for busy networks



TimeSync browser lab

Shreyansh, Vatsal and Aniketh

THE PROBLEM

One event can get two different timestamps

MACHINE A

`10:00:00.000`

MACHINE B

`10:00:00.100`

Same event, 100 ms apart

OUTAGE RESPONSE

Teams can chase the wrong cause

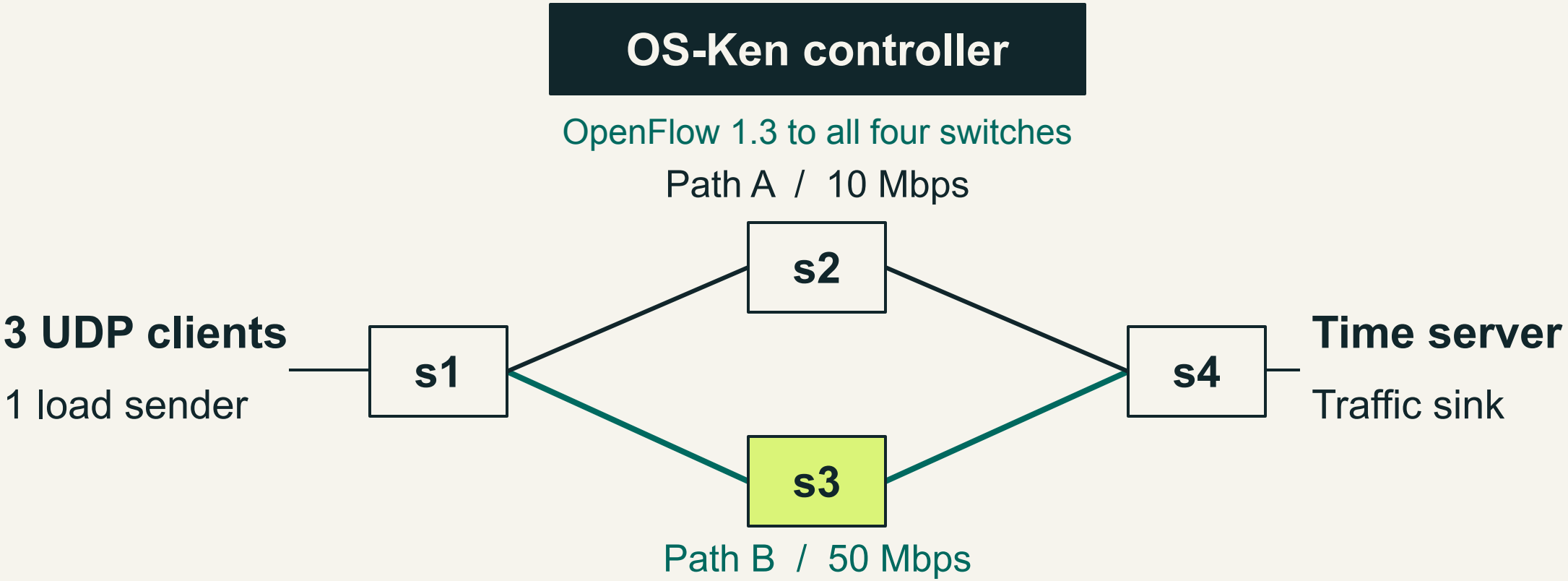
ACCESS CONTROL

Valid tokens can be rejected early

AUDIT TRAILS

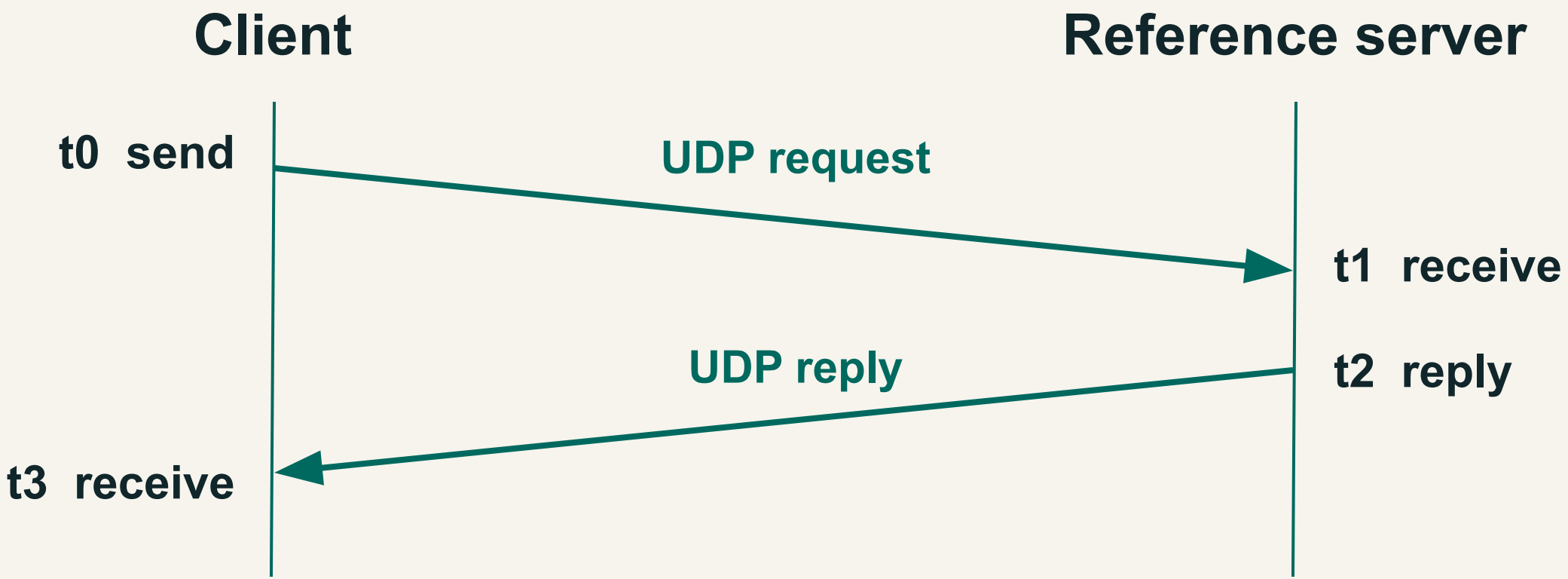
Event order can become disputed

Two paths lead to one time server



We used OS-Ken, a Ryu fork, to program the switches from Python.

One round trip gives us four timestamps



Four timestamps give us a clock correction

app/protocol.py

```
delay = (t3 - t0) - (t2 - t1)
offset = ((t1-t0)+(t2-t3))/2.0
```

The client adds the offset to its virtual clock.

The estimate works best when travel times are similar both ways.

Each datagram has a 35-byte format

app/protocol.py

```
PACKET_FORMAT = "!4sBHIddd"
```

The magic value “NTPS” identifies TimeSync datagrams.

“!” means big-endian, with the most significant byte first.

Magic	Type	Client ID	Sequence	t0	t1	t2
4 B	1 B	2 B	4 B	8 B	8 B	8 B

One UDP socket serves three clients

app/server.py / receive loop

```
t1 = time.time()
self.submit_request(sock, data,
                    client_addr, t1)
```

Validate first. Limit pending work to 128 requests.

Only a matching reply can adjust the clock

Type	Sequence	Client ID	Echoed t0

The sender address and all four fields must match.

Invalid replies leave the clock uncorrected.

UDP port 12300 identifies sync traffic

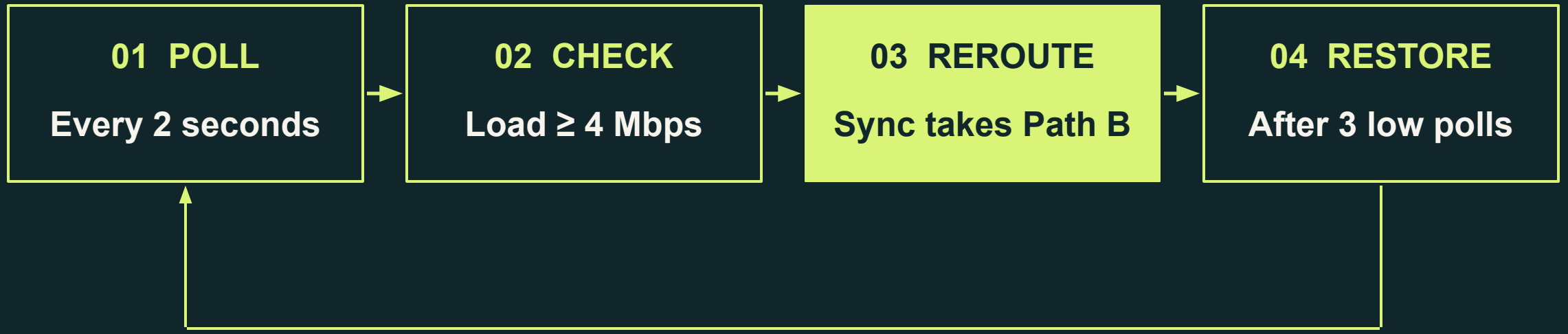
sdn/controller.py / s1 request match

```
m_fwd = parser.OFPMatch(eth_type=0x0800,
                        ip_proto=17,
                        udp_dst=self.sync_udp_port)
```

Requests match destination. Replies match source.

On Path A, sync gets queue 1 and background traffic is capped at 6 Mbps.

TimeSync reroutes when Path A gets busy



After restoration, sync uses Path A and polling continues.

Both policies face the same conditions

0–20 s

Normal

Sync traffic only

20–40 s

Congestion

12 Mbps offered load

40–60 s

Impairment

50 ms delay + 15 ms jitter

5 paired runs / 60 seconds per mode

We tested the code and the live network

94 → **6,400** → **PASS**

Python tests

Unit, integration, regression

UDP load attempts

32 concurrent clients

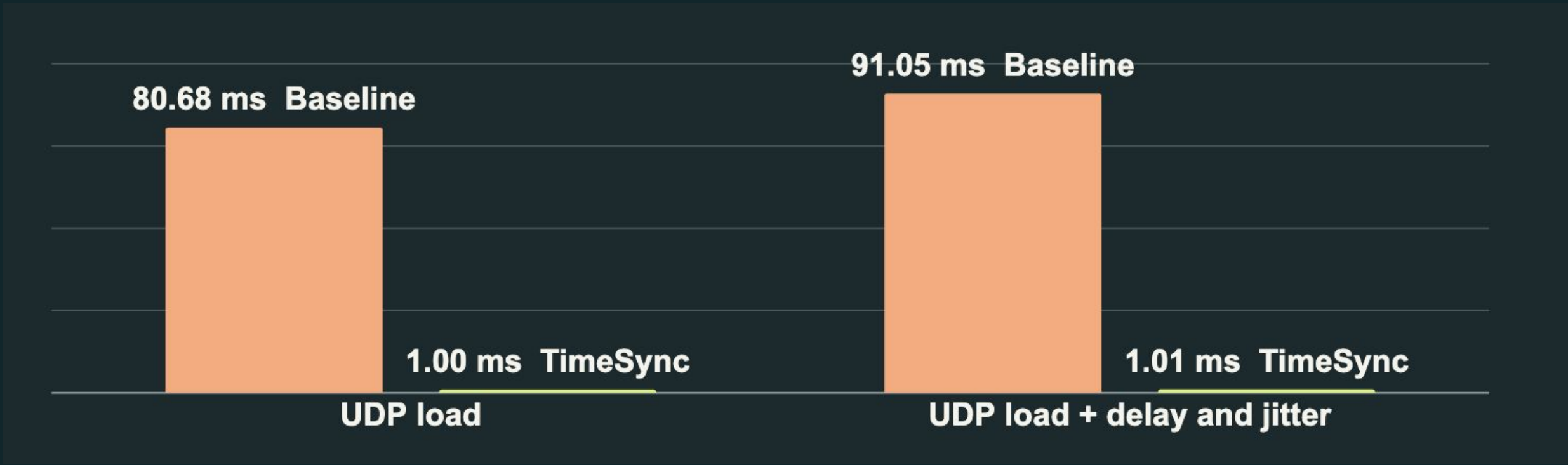
Automatic recovery

Fresh Linux VM run

Final checks: 4 November 2026. Recorded performance results remain unchanged.

TimeSync cut mean clock error under load

Mean post-attempt clock error under load



Five paired VM runs. Each bar averages the five run means for that condition.

MEASURED RESULTS

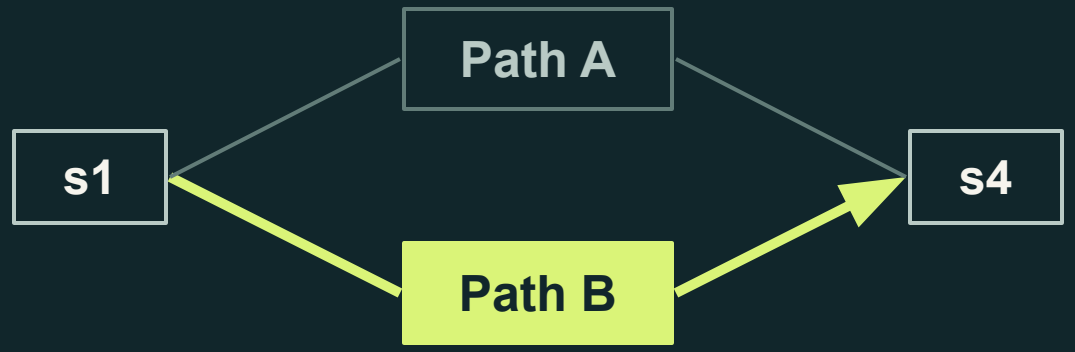
TimeSync recorded zero timeouts

Policy	Mean RTT per run	Timed-out attempts
Baseline	96–122 ms	393 / 1,753
TimeSync	22–23 ms	0 / 1,800

RTT uses successful replies. Timeout counts use all attempts.

Sync returned to Path A after load stopped

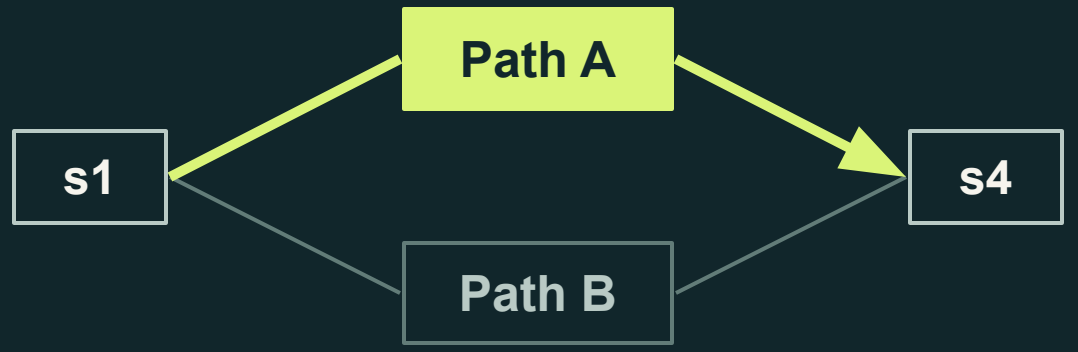
DURING LOAD



108

Sync requests counted on Path B

AFTER LOAD STOPPED



132 to 204

Path A request counter

Temporary rules cleared. All three clients received replies.

LIVE DEMO

The live run makes the route change visible

```
./timesync
```

4	5 / 6	9
Compare policies	Show results	Verify recovery

Scan for the repository

https://github.com/Annadata-Aniketh/Network-Time-Sync

THE TAKEAWAY

TimeSync kept clocks aligned under load

RESULT	About 1 ms vs 80+ ms mean error under load
METHOD	NTP-style timing over custom UDP, steered by SDN
LIMIT	We measured queueing and rerouting as one policy

Five paired Mininet VM runs